

Programación orientada a objetos

...

Algoritmos y Estructuras de Datos

¿Cómo trabajaría con horas?

¿Cómo trabajaría con horas?

- ¿Qué necesitaríamos para definir un tiempo específico?
- ¿Se debe usar tipos de datos determinados?
- Hay operaciones que sean distintas o que tengan restricciones distintas a las de, por ejemplo, un número entero?

Sobre Objetos, Clases y Métodos

- Una **clase** es una plantilla que define las características de un tipo de datos:
 - Variables, estructuras de datos, otras clases
 - Funciones asociadas
- Un **objeto** es una instancia de una clase.
- Los **métodos** son todas las funciones definidas para esa clase

```
>>> a = "objetos"  
>>> type(a)  
<class 'str'>
```



La variable ***a*** es una instancia de la clase ***string***

Sobre Objetos, Clases y Métodos

- Cada clase tiene características y comportamientos propios, más allá de que compartan o no las mismas funciones u operadores.

```
>>> entero = 123
>>> string = "objetos"
>>> string+string
'objetosobjetos'
>>> len(string)
7
>>> entero+entero
246
>>> len(entero)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: object of type 'int' has no len()
```

Sobre Objetos, Clases y Métodos

- Cada clase tiene características y comportamientos propios, más allá de que compartan o no las mismas funciones u operadores.

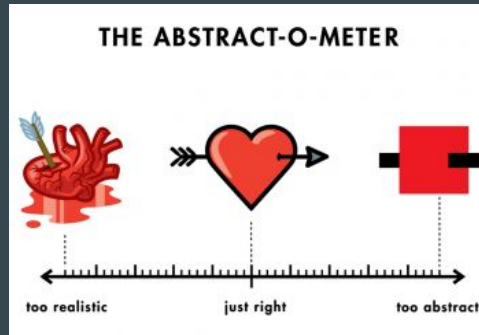
Operador “+”

```
>>> entero = 123
>>> string = "objetos"
>>> string+string
'objetosobjetos'
>>> len(string)
7
>>> entero+entero
246
>>> len(entero)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: object of type 'int' has no len()
```

Función “len”

Principios

- **Modularidad:** un sistema se compone de muchos componentes independientes que interactúan entre sí.
- **Encapsulación:** estos componentes deberían poder modificarse sin afectar otros componentes. Los detalles internos de un componente no deberían ser accesibles por otros componentes, objetos o funciones.
- **Abstracción:** intentar limitar un problema a sus partes más fundamentales.



Objetivos del uso de objetos

- **Robustez:** capaz de manejar entradas inesperadas.
- **Adaptabilidad:** capacidad de correr en distintas plataformas y permitir la implementación de nuevas características acordes a los cambios en el ambiente.
- **Reusabilidad:** un mismo bloque de código debería poder usarse en diferentes aplicaciones

Creando clases

- Para definir una nueva clase debemos definirla usando la palabra reservada **class**.
- Dentro de una clase primero se define el **constructor** con la función **__init__**.
- **self** hace referencia a la **instancia** de la clase que invoca algún método.

```
class Tiempo:
    def __init__(self, h, m, s):
        self._hora = h
        self._minuto = m
        self._segundo = s
```

```
# Código principal
t1 = Tiempo(17, 30, 40)
t2 = Tiempo(23, 59, 59)
print(t1._hora) # Devuelve "17"
print(t2._hora) # Devuelve "23"
```

Funciones

- Las funciones se definen con *def* dentro de la clase.
- Para acceder a los valores del objeto particular tenemos que usar la variable *self*

```
class Tiempo:
    def __init__(self, h, m, s):
        self._hora = h
        self._minuto = m
        self._segundo = s
    def devolver_minutos(self):
        return self._minuto
```

```
# Código principal
t1 = Tiempo(17, 30, 40)
t2 = Tiempo(23, 59, 59)
print(t1.devolver_minutos())
```

```
>>> t1 = Tiempo(17, 30, 40)
>>> print(t1.devolver_minutos())
30
```

Funciones

- Las funciones se definen con **def** dentro de la clase.
- Para acceder a los valores del objeto particular tenemos que usar la variable **self**

```
class Tiempo:
    def __init__(self, h, m, s):
        self._hora = h
        self._minuto = m
        self._segundo = s
    def devolver_minutos(self):
        return self._minuto
```

```
# Código principal
t1 = Tiempo(17, 30, 40)
t2 = Tiempo(23, 59, 59)
print(t1.devolver_minutos())
```

```
>>> a = "objetos"
>>> type(a)
<class 'str'>
>>> a.devolver_minutos()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'str' object has no attribute 'devolver_minutos'
```

**Las funciones son
propias de cada clase!**

Sintaxis (o “lo que dice el manual de buenas prácticas”)

- Dos guiones bajos al principio y al final:
 - Usos especiales en el lenguaje.
 - Define el comportamiento en **funciones** y **operadores** para una clase particular
 - Algunos ejemplos:
 - `__init__` para modificar “=”
 - `__len__` para modificar “len”
- Un guión bajo al principio:
 - **Partes ocultas** => Encapsulación
 - Convención

```
class Tiempo:
    def __init__(self, h, m, s):
        self._hora = h
        self._minuto = m
        self._segundo = s

t1 = Tiempo(17, 30, 40)
```

Polimorfismos en funciones binarias

- Algunos operadores tienen comportamientos distintos para distintas clases.
- Usar una misma entidad para representar diferentes operaciones en distintos escenarios se conoce como *polimorfismo*.
- Se definen como una función más, pero con nombres especiales.

Operator	Magic Method
+	<code>__add__(self, other)</code>
-	<code>__sub__(self, other)</code>
*	<code>__mul__(self, other)</code>
/	<code>__truediv__(self, other)</code>
//	<code>__floordiv__(self, other)</code>
%	<code>__mod__(self, other)</code>

Polimorfismos en funciones binarias

- Algunos operadores tienen comportamientos distintos para distintas clases.
- Usar una misma entidad para representar diferentes operaciones en distintos escenarios se conoce como *polimorfismo*.
- Se definen como una función más, pero con nombres especiales.

```
class Tiempo:
    # __init__ y demases

    def __sub__(self, other):
        h = self._hora - other._hora
        m = self._minutos - other._minutos
        s = self._segundos - other._segundos
        return Tiempo(h, m, s)

# Código principal
t1 = Tiempo(17, 30, 40)
t2 = Tiempo(20, 32, 59)
t3 = t2 - t1
```

Errores (o “lo que dice el manual de buenas prácticas”)

- Para incrementar la robustez de un código es necesario asegurar que, al crear un objeto, se utilicen los datos correctos.
- La función ***raise*** permite informar al usuario qué error cometió

Class	Description
Exception	A base class for most error types
AttributeError	Raised by syntax obj.foo, if obj has no member named foo
EOFError	Raised if “end of file” reached for console or file input
IOError	Raised upon failure of I/O operation (e.g., opening file)
IndexError	Raised if index to sequence is out of bounds
KeyError	Raised if nonexistent key requested for set or dictionary
KeyboardInterrupt	Raised if user types ctrl-C while program is executing
NameError	Raised if nonexistent identifier used
StopIteration	Raised by next(iterator) if no element; see Section 1.8
TypeError	Raised when wrong type of parameter is sent to a function
ValueError	Raised when parameter has invalid value (e.g., sqrt(-5))
ZeroDivisionError	Raised when any division operator used with 0 as divisor

Errores (o “lo que dice el manual de buenas prácticas”)

- Para incrementar la robustez de un código es necesario asegurar que, al crear un objeto, se utilicen los datos correctos.
- La función ***raise*** permite informar al usuario qué error cometió

```
>>> a = "objetos"
>>> type(a)
<class 'str'>
>>> a.devolver_minutos()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'str' object has no attribute 'devolver_minutos'
```


Errores (o “lo que dice el manual de buenas prácticas”)

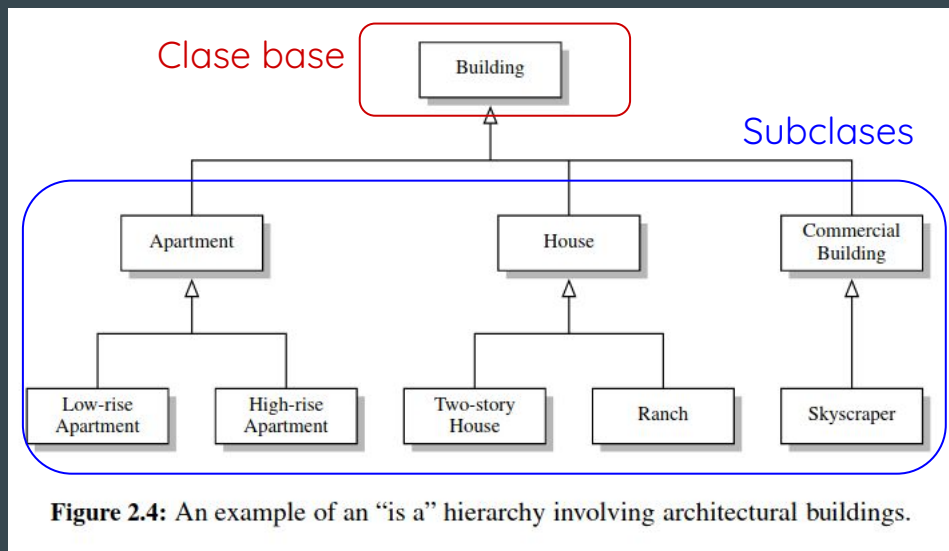
- Para incrementar la robustez de un código es necesario asegurar que, al crear un objeto, se utilicen los datos correctos.
- La función ***raise*** permite informar al usuario qué error cometió

```
class Tiempo:
    def __init__(self, h, m, s):
        if not isinstance(h, int):
            raise TypeError("Hora debe ser int")

        if h > 23 or h < 0:
            raise ValueError("Hora debe ser entre 0 y 23")
        self._hora = h
```

Herencia

- Permite definir nuevas clases a partir de clases ya definidas.
- Una subclase puede especializarse en un comportamiento existente, con métodos y variables nuevas, o expandir nuevos comportamientos.
- El mecanismo para definir una subclase se basa en el constructor *super*.



Ejemplo 1: registro de clientes

- Al momento de realizar la primer compra, se piden los siguientes datos:
 - Nombre
 - Apellido
 - Domicilio
 - Teléfono
 - Correo electrónico
- El local tiene un sistema de notificaciones por correo y teléfono, del cual los clientes puede optar por desuscribirse.
- Luego de realizar 5 compras en el local, los clientes pueden acceder a un descuento.
- Se requiere construir un programa que permita almacenar toda esta información para cada cliente nuevo.

Ejemplo 1: registro de clientes

```
class Cliente:  
    def __init__(self, n, a, d, c, t):  
        self._nombre = n  
        self._apellido = a  
        self._domicilio = d  
        self._correo = c  
        self._telefono = t  
        self._suscripto = True  
        self._compras = 1
```

Ejemplo 1: registro de clientes

```
class Cliente:
    def __init__(self, n, a, d, c, t):
        # Código del constructor

    def agregar_compra (self):
        self._compras += 1

    def descuento (self):
        if self._compras > 5:
            return True
        else:
            return False

    def desuscribir (self):
        self._suscripto = False
```

Ejemplo 2: personas en la universidad

- Se quiere construir un programa que guarde la información de las distintas personas de la universidad.
- Hay información que todas las personas poseen (nombre, apellido, DNI, etc).
- También hay información propia de algunos grupos:
 - Alumnos: carrera(s) inscripto, año de ingreso, materias aprobadas, etc
 - Docente: materia(s) dictada(s), título universitario, horas trabajadas en el mes, etc
 - No docente: área en que trabaja, rol, horas trabajadas en el mes, etc
- Podemos pensar este problema como una serie de grupos que comparten **información base**, pero que tienen **datos específicos** de cada grupo.

Ejemplo 2: personas en la universidad

```
class Persona:  
    def __init__(self, n, a, d):  
        self._nombre = n  
        self._apellido = a  
        self._dni = d
```

Ejemplo 2: personas en la universidad

```
class NoDocente(Persona):  
    def __init__(self, n, a, d):  
        super().__init__(n, a, d)  
        self._horas_mes = 0  
        self._area = ""  
  
    def cambiar_area(self, a):  
        self._area = a  
  
    def contabilizar_horas(self, h):  
        self._horas_mes += h  
  
    def horas_trabajadas(self):  
        return self._horas_mes
```


Ejemplo 2: personas en la universidad

```
class Docente(Persona):
    def __init__(self, n, a, d):
        super().__init__(n, a, d)
        self._materias = []
        self._titulo = ""
        self._cargo = ""
        self._horas_mes = 0

    def agregar_titulo(self, titulo):
        self._titulo = titulo

    def agregar_cargo(self, cargo):
        self._cargo = cargo

    def agregar_materia(self, materia):
        self._materias.append(materia)

    def cantidad_materias(self):
        return len(self._materias)
```

Ejemplo 2: personas en la universidad

```
class Alumno(Persona):  
    def __init__(self, n, a, d, i, c):  
        super().__init__(n, a, d)  
        self._ingreso = i  
        self._carreras = [c]  
  
    def agregar_carrera(self, c):  
        self._carreras.append(c)  
  
    def cambiar_carrera(self, ant, nueva):  
        self._carreras.remove(ant)  
        self._carreras.append(nueva)
```

Bibliografía

- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). Data structures and algorithms in Python. Capítulo 1